

STRATEGIC PRODUCT & MARKET INTELLIGENCE

TOP 10 VALUE-ROCKET GROWTH STRATEGY

How Jupiter Becomes the Default File Manager for Power Users,
Creators and Privacy-Conscious Android Users

Smart, privacy-first Android file manager with real LAN/cloud backends and on-device AI

€1.4M-€3.2M

Target Valuation

8-10x

Value Multiple

10

Growth Initiatives

Jupiter — Smart, privacy-first Android file manager with real LAN/cloud backends and on-device AI

Prepared: 2026-06-30 | Confidential

Baseline Valuation: €140k-€320k → Target Valuation: €1.4M-€3.2M

EXECUTIVE SUMMARY

What this report contains

A ranked set of high-leverage growth initiatives for Jupiter, each scored by expected valuation impact, with a ready-to-paste implementation prompt and a regeneration prompt for iterative deepening.

How to use it

Read the matrix on the final page for the overview. Each ranked entry stands alone — pick any to deep-dive. The Implementation Prompt section is engineered to be pasted directly into an AI coding assistant inside the codebase.

Methodology: market multiples sourced from public valuation databases (PitchBook, Crunchbase, SaaS Capital Index), competitor pricing pages, and analyst reports (Gartner, IDC, Forrester). Valuation impact is range-based and triangulated across replacement-cost, comparable-ARR-multiple, and strategic-premium lenses.

TABLE OF CONTENTS

#	Initiative	Est. Impact
1	Jupiter Pro — Monetization & Entitlement Engine	+€350k-€700k
2	Real Cloud Storage — Google Drive / Dropbox / OneDrive (OAuth + REST)	+€220k-€420k
3	Localization & Global Reach (12+ languages)	+€120k-€260k
4	AI Pro Suite (Claude) — Summarize, Smart-Rename, Auto-Organize, Semantic Search	+€160k-€320k
5	Home-screen Widgets, App Shortcuts & Quick-Settings Tile	+€90k-€180k
6	Privacy-first Opt-in Product Analytics & Crash Reporting	+€80k-€170k
7	Activation & Retention Loop (Onboarding Funnel, What's-New, Rating, Re-engagement)	+€90k-€190k
8	High-performance Search Index (Room FTS) + Saved Searches + Content Search	+€110k-€220k
9	Personalization & Theming (Material You Accent, Icon/Density Themes, Custom Home)	+€70k-€150k
10	Share & Interop Hub (Receive "Save to Jupiter", Quick-Share, SAF DocumentsProvider)	+€100k-€200k

TOTAL COMBINED POTENTIAL

+€1.39M-€2.81M

RANK 1**Jupiter Pro — Monetization & Entitlement Engine****POINT 2 — DESCRIPTION & MARKET EVIDENCE**

This is the single biggest valuation lever for Jupiter. Today the app is a feature-complete, pre-revenue MVP: 33,932 lines of Kotlin shipping real LAN/SMB/SFTP/FTP/WebDAV backends, an EncryptedFile vault, a Wi-Fi server and a Claude AI assistant — but with no way to capture value. A freemium Pro tier built on Google Play Billing converts that engineering into a revenue-generating product, which is exactly what re-rates a build-cost-plus-IP asset (the €140k–€320k Section 9 baseline) into a multiple-of-revenue business. The design principle is non-regression: introduce an EntitlementManager that defaults every Feature to UNLOCKED until a real billing product is configured, so nothing that already works for today's users ever breaks.

The market validates the model directly. Solid Explorer ships a 14-day trial then a one-time unlock; X-plore and FX File Explorer both run free-with-paid-Pro; MiXplorer and Total Commander monetize add-ons. Files by Google stays free because Google monetizes elsewhere — leaving a clear gap for a privacy-first paid app with no ads and no dark patterns. A consumer utility converting even 2–4% of installs to a €4.99–€9.99 lifetime unlock, plus an optional subscription for the AI Suite, is a well-understood pattern on Google Play. Crucially, gating premium surfaces (Vault, NAS/remote, AI Suite, dual-pane, advanced cleanup) behind a tier signals to an acquirer that the revenue rails exist and are reversible per-feature.

Technical approach for this Kotlin/Compose/Hilt codebase: add a new core/entitlement package with an EntitlementManager (Hilt singleton), a Feature enum (VAULT, REMOTE_NAS, AI_SUITE, DUAL_PANE, ADVANCED_CLEANUP) and a Tier model exposed as a StateFlow. Add a feature/billing package wrapping com.android.billingclient:billing-ktx in a BillingClient wrapper, with a Compose PaywallScreen and an UpgradeViewModel that queries products, launches the purchase flow and acknowledges entitlements. Persist the unlocked state via the existing data/preferences DataStore pattern (SettingsDataStore.kt). Add lightweight gate checks — a single isUnlocked(Feature) call — at premium entry points in feature/vault/VaultViewModel.kt, feature/cloud/NasConnectionsViewModel.kt, feature/ai/AiAssistantViewModel.kt and the feature/browser dual-pane path, each falling through to UNLOCKED until billing is live.

POINT 3 — IMPLEMENTATION PROMPT

```
Create core/entitlement/EntitlementManager.kt
@Singleton Hilt class, injected app-wide
fun isUnlocked(feature: Feature): Boolean
val tier: StateFlow<Tier> // FREE / PRO
Default: every Feature UNLOCKED until a billing product is configured

Create core/entitlement/Feature.kt
enum Feature { VAULT, REMOTE_NAS, AI_SUITE, DUAL_PANE, ADVANCED_CLEANUP }

Create core/entitlement/Tier.kt // sealed: Free, Pro(purchaseToken)

Create feature/billing/BillingClientWrapper.kt
Wrap com.android.billingclient:billing-ktx
queryProductDetails, launchBillingFlow, acknowledgePurchase
Map purchases -> EntitlementManager.tier

Create feature/billing/PaywallScreen.kt (Compose, Material 3)
Create feature/billing/UpgradeViewModel.kt

Persist unlock state via data/preferences/SettingsDataStore.kt
```

```
Add gate checks (fall through to UNLOCKED until billing live):
feature/vault/VaultViewModel.kt          -> Feature.VAULT
feature/cloud/NasConnectionsViewModel.kt -> Feature.REMOTE_NAS
feature/ai/AiAssistantViewModel.kt      -> Feature.AI_SUITE
feature/browser (dual-pane path)         -> Feature.DUAL_PANE
feature/cleanup/SmartMergeViewModel.kt   -> Feature.ADVANCED_CLEANUP

Gradle: add com.android.billingclient:billing-ktx
Register Pro product + subscription in Play Console
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Business model	Pre-revenue -> revenue-generating
Valuation re-rating	Build-cost+IP -> revenue multiple
Install->Pro conversion	2-4% (utility freemium benchmark)
Regression risk	Zero (default UNLOCKED)

ESTIMATED VALUATION IMPACT

+€350k-€700k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter (com.jupiter.filemanager, native Kotlin/Compose/Hilt Android file manager) and produce a Jupiter Pro monetization & entitlement plan. Cover: (1) why freemium/Play Billing is the #1 pre-revenue valuation lever, citing Solid Explorer/X-plore/FX/Files by Google models, (2) core/entitlement Entitlement Manager + Feature enum + Tier with default-UNLOCKED non-regression design, (3) feature/billing BillingClient wrapper + PaywallScreen + UpgradeViewModel, gate checks at vault/cloud/ai/dual-pane/cleanup, (4) valuation re-rating impact, (5) regeneration meta-prompt. Point 1-5 structure.

RANK 2 Real Cloud Storage — Google Drive / Dropbox / OneDrive (OAuth + REST)

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter already ships a Cloud Hub scaffold (feature/cloud/CloudHubScreen.kt, CloudHubViewModel.kt) and a CloudAccount domain model whose CloudProvider enum already enumerates GOOGLE_DRIVE, DROPBOX, ONEDRIVE, ICLOUD, BOX and WEBDAV. Turning that scaffold into live cloud browsing and transfer is the second-biggest lever because it closes the most visible feature gap against the category leaders and unlocks the exact buyers who pay: people with files spread across personal cloud accounts. This is additive — the LAN backends and the remote abstraction already exist and work — so it extends a proven pattern rather than inventing new architecture.

Competitively this is table stakes done well. Solid Explorer's headline differentiator is its broad cloud provider list; FX, X-plore and CX File Explorer all advertise Drive/Dropbox/OneDrive. Files by Google deliberately omits third-party clouds. A privacy-first manager that browses and transfers across Drive, Dropbox and OneDrive — with OAuth tokens stored in the existing encrypted CredentialStore and no telemetry — is a direct, defensible answer to 'why not just use Solid Explorer'. Cloud connectivity is also the most common single reason users pay for a file manager, which feeds directly into the Pro tier (initiative #1).

Technical approach: mirror exactly how SMB/SFTP/FTP/WebDAV are wired in data/remote. Add a new data/cloud package with AppAuth-based OAuth (Authorization Code + PKCE) and per-provider REST clients over OkHttp (Drive v3, Dropbox v2, Microsoft Graph). Each provider implements the existing domain/remote/RemoteFileSource interface (type/testConnection/list/download), so the Cloud Hub, transfer queue (data/transfer) and browser reuse the same code paths as NAS. Wire accounts through ConnectionRepositoryImpl.kt and the CloudAccount model; store refresh tokens via data/remote/CredentialStore.kt. CloudHubViewModel.kt switches from placeholder accounts to live RemoteFileSource calls.

POINT 3 — IMPLEMENTATION PROMPT

```
Reuse domain/remote/RemoteFileSource.kt (type/testConnection/list/download)
Reuse domain/model/CloudAccount.kt (CloudProvider already has DRIVE/DROPBOX/ONEDRIVE)

Create data/cloud/oauth/CloudOAuthManager.kt
  AppAuth Authorization Code + PKCE
  Store refresh tokens via data/remote/CredentialStore.kt (encrypted)

Create data/cloud/GoogleDriveFileSource.kt (Drive v3 REST, OkHttp)
Create data/cloud/DropboxFileSource.kt (Dropbox v2 REST, OkHttp)
Create data/cloud/OneDriveFileSource.kt (Microsoft Graph, OkHttp)
  each implements RemoteFileSource (mirrors SmbFileSource.kt etc.)

Wire into data/connection/ConnectionRepositoryImpl.kt
Register sources in data/remote/RemoteSourceProviderImpl.kt

feature/cloud/CloudHubViewModel.kt -> live accounts via RemoteFileSource
feature/cloud/CloudHubScreen.kt -> connect/disconnect, browse, transfer
Reuse data/transfer for upload/download queue

Gradle: add net.openid:appauth + okhttp (already present)
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Feature parity	Closes #1 gap vs Solid Explorer/FX/X-plore
Pro upsell driver	Cloud = top paid-conversion trigger
Reuse	100% of existing RemoteFileSource abstraction
New providers	Drive + Dropbox + OneDrive live

ESTIMATED VALUATION IMPACT

+€220k-€420k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design real cloud storage (Google Drive/Dropbox/OneDrive) on top of the existing Cloud Hub scaffold. Cover: (1) competitive gap vs Solid Explorer/FX/Files by Google and pay-conversion rationale, (2) data/cloud AppAuth PKCE OAuth + per-provider OkHttp REST sources implementing domain/remote/RemoteFileSource, mirroring SMB/SFTP/FTP/WebDAV in data/remote, (3) wiring via ConnectionRepositoryImpl + CloudAccount + CredentialStore, CloudHubViewModel going live, (4) parity/upsell valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

RANK 3**Localization & Global Reach (12+ languages)****POINT 2 — DESCRIPTION & MARKET EVIDENCE**

Jupiter's UI strings are currently hardcoded across feature/* Compose screens. Externalizing them to res/values/strings.xml and shipping res/values- translations (hu, de, es, fr, pt, it, nl, pl, ru, tr, ja, ko) is the highest-ROI install-base multiplier available to a global consumer utility, and it is the lowest-risk initiative on this list: it touches strings only, so there is no behavioral regression surface. A file manager is intrinsically global — every Android user has files — so removing the English-only barrier directly widens the addressable install base.

The evidence is in the leaders' listings. Files by Google ships in dozens of languages; Solid Explorer and X-plore are heavily community-translated; localized store listings measurably lift install conversion in non-English markets (Google Play's own localization guidance). For a privacy-first app, localization is also a trust signal in regions (DACH, France, Japan) where data-handling expectations are high. Because Jupiter is pre-revenue, a larger and more global install base is the raw material every later monetization initiative compounds on.

Technical approach: extract literals from feature/* Compose screens (HomeScreen.kt, SearchScreen.kt, VaultScreen.kt, SettingsScreen.kt, CloudHubScreen.kt and the rest) into app/src/main/res/values/strings.xml, replacing them with stringResource(R.string....) calls. Add per-locale res/values-/strings.xml files. Keep parameterized strings as proper Android format resources for counts and sizes. The work is mechanical and parallelizable, integrates cleanly with the existing Compose/Material 3 UI, and can be partly bootstrapped with machine translation then human-reviewed for the priority locales.

POINT 3 — IMPLEMENTATION PROMPT

```
Create app/src/main/res/values/strings.xml
  Extract hardcoded literals from feature/* Compose screens:
    feature/home/HomeScreen.kt, feature/search/SearchScreen.kt,
    feature/vault/VaultScreen.kt, feature/settings/SettingsScreen.kt,
    feature/cloud/CloudHubScreen.kt, feature/cleanup/*Screen.kt, ...
  Replace literals with stringResource(R.string.key)

Create per-locale resource dirs:
  res/values-hu/strings.xml, res/values-de/strings.xml,
  res/values-es/, -fr/, -pt/, -it/, -nl/, -pl/, -ru/, -tr/, -ja/, -ko/

Use plurals/format args for counts and file sizes
Localize Play Store listing per priority market
Strings-only change: zero behavioral regression
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Addressable install base	English-only -> 12+ locales
Listing conversion lift	Higher in non-English markets
Regression risk	Minimal (strings only)
Trust signal	DACH/FR/JP privacy-sensitive markets

ESTIMATED VALUATION IMPACT

+€120k-€260k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design a localization plan for 12+ languages. Cover: (1) install-base multiplier rationale for a global utility, citing Files by Google/Solid Explorer localization and Play listing conversion, (2) extract hardcoded literals from feature/* Compose screens into res/values/strings.xml with stringResource refactors, add res/values-<lang> for hu/de/es/fr/pt/it/... , plurals/format handling, (3) regression-safe strings-only scope, (4) addressable-base valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

RANK 4
AI Pro Suite (Claude) — Summarize, Smart-Rename, Auto-Organize, Semantic Search

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter already ships a real, key-gated Claude assistant: feature/ai/AnthropicAiAssistant.kt implements the AiAssistant interface (smart-rename suggestions, suggestions surfaced as AiSuggestion), with NoOpAiAssistant.kt as the safe fallback when no key is configured. Expanding this into a four-pillar AI Pro Suite — document/folder Summarize, AI Smart-Rename, one-tap Auto-Organize suggestions, and natural-language Semantic Search — turns a competent feature into the marquee differentiator no other Android file manager has. This is what makes Jupiter narratively unique to an acquirer and is the natural anchor of the Pro/subscription tier (initiative #1).

No mainstream Android file manager — Solid Explorer, X-plore, FX, Files by Google — ships on-device-gated, bring-your-own-key LLM actions. That is a genuine open lane. The product principle that makes it safe and trustworthy is 'confirm-before-apply, no hallucination': every AI action proposes a concrete change (a rename, a move plan, a summary) that the user explicitly approves before anything touches the filesystem. For a privacy-first brand, the Anthropic-key-gated design (no key, no calls; user's own key) is itself the trust story — the opposite of opaque cloud AI.

Technical approach: extend the existing feature/ai/AiAssistant.kt interface with summarize(file/folder), proposeOrganization(files) and semanticQuery(query) returning structured, confirmable plans; implement them in AnthropicAiAssistant.kt using the Anthropic Messages API with Claude tool-use, keeping NoOpAiAssistant.kt in lockstep. Surface results through AiAssistantScreen.kt / AiAssistantViewModel.kt and add a universal-action sheet hook so any selected file routes into an AI action. Wire Semantic Search into feature/search/SearchViewModel.kt (ranking on top of results) and Auto-Organize into feature/cleanup. Every action is a preview the user applies via the existing file operation pipeline (data/file).

POINT 3 — IMPLEMENTATION PROMPT

```
Extend feature/ai/AiAssistant.kt interface:
suspend fun summarize(item: FileItem): AppResult<String>
suspend fun proposeOrganization(items: List<FileItem>): AppResult<OrganizePlan>
suspend fun semanticQuery(query: String, candidates): AppResult<List<RankedHit>>
(keep existing smart-rename)

Implement in feature/ai/AnthropicAiAssistant.kt
  Anthropic Messages API + Claude tool-use
  Confirm-before-apply: return plans, never auto-mutate
Mirror no-op paths in feature/ai/NoOpAiAssistant.kt

feature/ai/AiAssistantScreen.kt + AiAssistantViewModel.kt: 4 actions UI
Universal-action sheet hook: route selected file -> AI action

feature/search/SearchViewModel.kt: semantic ranking layer over results
feature/cleanup: Auto-Organize suggestions surface
Apply approved changes through data/file operation pipeline
Gate behind Feature.AI_SUITE (entitlement, initiative #1)
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Category differentiation	Only AI-native Android file manager

Pro/subscription anchor	Marquee paid tier feature
Privacy posture	BYO-key, confirm-before-apply, no hallucination
Reuse	Extends existing AnthropicAiAssistant

ESTIMATED VALUATION IMPACT

+€160k-€320k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design an AI Pro Suite (Claude) extending the existing key-gated AnthropicAiAssistant . Cover: (1) why AI-native is a unique differentiator vs Solid Explorer/X-plore/FX/Files by Google and anchors the Pro tier, (2) extend feature/ai/AiAssistant.kt with summarize/proposeOrganization/semanticQuery , implement in AnthropicAiAssistant.kt via Anthropic Messages API + tool-use, NoOp mirror, AiAssistantScreen/ViewModel + universal-action sheet + SearchViewModel semantic ranking + cleanup auto-organize, confirm-before-apply through data/file, (3) BYO-key privacy posture, (4) differentiation/upsell valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

RANK 5

Home-screen Widgets, App Shortcuts & Quick-Settings Tile

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter currently lives only inside its own activity. Surfacing it on the home screen and in system UI — a Glance home-screen widget (storage gauge + quick access), dynamic and static app shortcuts (Search / Cleanup / Vault), and a Quick-Settings tile for one-tap cleanup — drives daily active use and retention without any new core feature. These are pure additions through new components and the manifest, so there is no regression risk, and they make Jupiter feel like an always-present system utility rather than an app the user has to remember to open.

This is a proven retention pattern in the category. Files by Google surfaces a storage cleanup card and notifications; Samsung's My Files exposes shortcuts; widgets are a well-documented DAU driver on Android because they keep the app on the most valuable real estate on the device. For a utility whose core jobs (free up space, find a file, open the vault) are quick and recurring, putting those one tap away from the home screen materially lifts engagement — and engagement is the metric every later monetization step compounds on.

Technical approach: add a new app/widget package using androidx.glance:glance-appwidget to build a GlanceAppWidget plus its receiver, reading storage state from the existing StorageAnalyticsRepository so the gauge is real. Add an app/shortcuts package providing static shortcuts in res/xml plus dynamic ShortcutManager shortcuts for Search, Cleanup and Vault. Add an app/tile package with a TileService for one-tap cleanup that reuses feature/cleanup logic. Register the widget receiver, shortcuts and TileService in AndroidManifest.xml. Glance is Compose-native, so it fits the existing UI stack with no new paradigm.

POINT 3 — IMPLEMENTATION PROMPT

```
Create app/widget/JupiterGlanceWidget.kt (GlanceAppWidget)
    Storage gauge + quick-access buttons
    Read state from domain StorageAnalyticsRepository
Create app/widget/JupiterWidgetReceiver.kt (GlanceAppWidgetReceiver)

Create app/shortcuts/JupiterShortcuts.kt
    Static shortcuts via res/xml/shortcuts.xml (Search/Cleanup/Vault)
    Dynamic ShortcutManager shortcuts (recent locations)

Create app/tile/CleanupTileService.kt (TileService)
    One-tap cleanup; reuse feature/cleanup logic

AndroidManifest.xml: register receiver, <meta-data> shortcuts, TileService
Gradle: add androidx.glance:glance-appwidget
Pure additions: no regression to existing screens
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
DAU / retention	Always-present home-screen surface
Time-to-task	One tap to Cleanup/Search/Vault
Regression risk	Zero (additive components)
Reuse	StorageAnalyticsRepository + feature/cleanup

ESTIMATED VALUATION IMPACT

+€90k-€180k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design home-screen widgets, app shortcuts and a Quick-Settings tile. Cover: (1) DAU/r etention rationale citing Files by Google/Samsung My Files surfacing, (2) app/widget Glance GlanceAppWidg et+receiver reading StorageAnalyticsRepository, app/shortcuts static+dynamic ShortcutManager (Search/Clea nup/Vault), app/tile TileService reusing feature/cleanup, manifest registration, add androidx.glance:glan ce-appwidget, (3) additive zero-regression scope, (4) engagement valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

RANK 6**Privacy-first Opt-in Product Analytics & Crash Reporting****POINT 2 — DESCRIPTION & MARKET EVIDENCE**

Jupiter cannot improve activation, retention or conversion if it cannot measure them — yet bolting on a standard analytics SDK would betray the privacy-first promise that is its core differentiator. The resolution is a vendor-neutral, opt-in (default OFF), no-PII analytics and crash abstraction: a thin Analytics interface with a NoOpAnalytics implementation as the default, behind an explicit opt-in gate the user controls in settings. This de-risks the asset for an acquirer (who needs funnel and stability data in diligence) while keeping the no-ads, no-dark-patterns brand intact.

The market reality is that Files by Google, Solid Explorer and most competitors collect analytics by default; Jupiter's stance can instead be 'measurement only with your explicit consent, never any personal data, no third-party trackers'. That stance is itself marketable and aligns with GDPR-by-design expectations in Jupiter's strongest privacy-sensitive markets. Crucially the abstraction is pluggable: today it ships NoOp (literally measuring nothing until opted in), but the seam lets a privacy-respecting sink (or self-hosted endpoint) be added later without touching call sites.

Technical approach: add a new core/analytics package with an Analytics interface, an AnalyticsEvent model (typed funnel events, no free-form PII), a NoOpAnalytics default implementation provided via Hilt, and an opt-in gate that reads consent from data/preferences. Add a single privacy toggle in feature/settings (SettingsScreen.kt / SettingsViewModel.kt) defaulting to OFF. Instrument key funnel points — onboarding completion, first browse, vault unlock, cloud connect, AI action, Pro paywall view/purchase — with typed event calls that are no-ops until consent is granted. Crash reporting follows the same opt-in, no-PII contract.

POINT 3 — IMPLEMENTATION PROMPT

```
Create core/analytics/Analytics.kt (interface: track(event), setEnabled)
Create core/analytics/AnalyticsEvent.kt (typed events, NO free-form PII)
Create core/analytics/NoOpAnalytics.kt (default Hilt binding)
Create core/analytics/AnalyticsConsentGate.kt (reads opt-in)

feature/settings/SettingsScreen.kt + SettingsViewModel.kt:
  Privacy toggle, DEFAULT OFF, persisted via data/preferences

Instrument typed funnel events (no-op until consent):
  onboarding_completed, first_browse, vault_unlocked,
  cloud_connected, ai_action_used, paywall_viewed, pro_purchased

Crash reporting: same opt-in + no-PII contract, pluggable sink
Default ships NoOp: literally measures nothing until opted in
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Measurability	Activation/retention/conversion visible
Acquirer diligence	Funnel + stability data on demand
Privacy posture	Opt-in default OFF, no PII, no trackers
Default behavior	NoOp (zero data) until consent

ESTIMATED VALUATION IMPACT

+€80k-€170k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design privacy-first opt-in analytics & crash reporting. Cover: (1) why measurement is needed yet must not break the privacy-first promise, contrast vs default-collection competitors, (2) core/analytics Analytics interface + AnalyticsEvent + NoOpAnalytics default + consent gate via data/preferences, feature/settings toggle default OFF, typed funnel instrumentation no-op until consent, crash reporting same contract, (3) GDPR-by-design posture, (4) diligence/measurability valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

RANK 7 Activation & Retention Loop (Onboarding Funnel, What's-New, Rating, Re-engagement)

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter has an onboarding flow (feature/onboarding/OnboardingScreen.kt) and a home screen, but no closed loop that turns a first launch into an activated, returning, rating user. Tightening that loop — instrumented onboarding completion, a What's-New sheet on version bump, an in-app review prompt fired at the right moment, and an opt-in WorkManager re-engagement notification (e.g. 'X GB reclaimable') — is a high-leverage, additive growth play. It directly lifts the two metrics that compound under every monetization initiative: activation rate and 7-/30-day retention.

These are the standard, proven mechanics of successful consumer apps, and Files by Google is the obvious comparable: it nudges users with cleanup notifications and storage cards to drive return visits. Jupiter already has the substrate for the privacy-respecting version of this — a WorkManager automation system and an AppStateDataStore — so re-engagement can be strictly opt-in and value-led ('you can reclaim 3.2 GB') rather than spammy. Google Play's in-app review API also captures ratings at the peak-satisfaction moment, which lifts store rating and therefore organic install conversion.

Technical approach: instrument onboarding completion in feature/onboarding (writing a flag to data/preferences/AppStateDataStore.kt) and analytics (initiative #6). Add a new feature/whatsnew package with a Compose sheet that shows on detected version-code bump, gated by a last-seen version in AppStateDataStore. Add com.google.android.play:review-ktx and trigger the in-app review request after a positive action (e.g. a successful cleanup). Reuse the existing WorkManager automation to schedule an opt-in re-engagement notification computed from StorageAnalyticsRepository, defaulting OFF and respecting the analytics consent gate.

POINT 3 — IMPLEMENTATION PROMPT

```
feature/onboarding: write completion flag + emit analytics event
  Persist via data/preferences/AppStateDataStore.kt

Create feature/whatsnew/WhatsNewSheet.kt (Compose)
  Show on version-code bump; track last-seen in AppStateDataStore
  Surface from feature/home/HomeScreen.kt

In-app review: add com.google.android.play:review-ktx
  Trigger after a positive action (successful cleanup/transfer)

Re-engagement notification (opt-in, default OFF):
  Reuse WorkManager automation (data/automation)
  Compute 'X GB reclaimable' from StorageAnalyticsRepository
  Respect analytics/notification consent
Additive: no change to existing browse/file flows
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Activation rate	Instrumented onboarding completion
7-/30-day retention	What's-New + re-engagement uplift
Store rating	In-app review at peak satisfaction
Posture	Re-engagement opt-in, value-led, default OFF

ESTIMATED VALUATION IMPACT

+€90k-€190k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design an activation & retention loop. Cover: (1) why closing the loop lifts activation and 7/30-day retention, Files by Google comparable, (2) instrument onboarding completion via AppStateD ataStore + analytics, feature/whatsnew Compose sheet on version bump, com.google.android.play:review-ktx in-app review after positive action, opt-in WorkManager re-engagement notification from StorageAnalyticsR epository default OFF, (3) additive privacy-respecting scope, (4) activation/retention valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

High performance Search Index (Room FTS) + Saved Searches + Content Search

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter's current search (feature/search/SearchViewModel.kt) walks the filesystem on demand — correct, but slow on large storage and unable to do instant or content search. Backing search with a Room FTS4 index, built incrementally via WorkManager, turns search into an instant, cross-storage capability and gives Jupiter a genuine performance moat. Adding saved/recent searches and opt-in text-content search rounds it into a feature power users specifically choose a file manager for. It is additive: the index sits alongside the existing SearchViewModel, which can fall back to filesystem walk until the index is warm.

Search quality is a primary axis on which power-user file managers compete. X-plore and Solid Explorer differentiate on fast, deep search; Files by Google keeps search deliberately shallow. An FTS-backed instant search across every volume, plus content search inside text files, is exactly the capability that converts a 'good enough' manager into a daily driver — and instant search is a tangible, demonstrable wow moment in store screenshots and reviews. Saved searches additionally create the kind of stored user state that increases switching cost and stickiness.

Technical approach: add a new data/search package with a Room database, a FileIndexEntity, an FTS4 virtual table, and an indexer Worker that incrementally populates and updates the index via WorkManager (the app already uses WorkManager for automation). Expose it through a domain repository so feature/search/SearchViewModel.kt queries the index first and only falls back to a live walk when needed. Persist saved/recent searches in the same store and surface them in feature/search/SearchScreen.kt. Opt-in content search indexes extracted text for supported file types, gated and clearly disclosed for privacy. Add the androidx.room dependencies.

POINT 3 — IMPLEMENTATION PROMPT

```
Create data/search/JupiterSearchDatabase.kt (Room)
Create data/search/FileIndexEntity.kt + FTS4 virtual table (FileIndexFts)
Create data/search/FileIndexer.kt (WorkManager Worker, incremental)
    Build + update index across all storage volumes
Create data/search/SavedSearchEntity.kt (saved/recent searches)

Add domain repository (e.g. SearchIndexRepository) over the DB

feature/search/SearchViewModel.kt:
    Query FTS index first; fall back to filesystem walk if cold
    Saved/recent searches; opt-in content search toggle
feature/search/SearchScreen.kt: instant results + saved searches UI

Opt-in text-content search: clearly disclosed, privacy-gated
Gradle: add androidx.room (runtime, ktx, compiler)
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Search latency	Filesystem walk -> instant FTS lookup
Power-user moat	Content + saved search vs Files by Google
Stickiness	Saved searches = stored user state
Reuse	Additive alongside existing SearchViewModel

ESTIMATED VALUATION IMPACT

+€110k-€220k**POINT 5 — REGENERATION PROMPT**

Analyze Jupiter and design a Room FTS search index with saved searches and content search. Cover: (1) why instant/content search is a power-user moat vs X-plore/Solid Explorer/Files by Google, (2) data/search Room DB + FileIndexEntity + FTS4 + WorkManager indexer Worker, domain repository, SearchViewModel querying index first with filesystem fallback, saved/recent searches + opt-in content search in SearchScreen, add androidx.room, (3) additive non-regression scope, (4) performance/stickiness valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

BANK 9 Personalization & Theming (Material You Accent, Icon/Density Themes, Custom Home)

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter already has a JupiterTheme system (ui/theme/Theme.kt, Color.kt). Building on it to let users tailor the look — an accent-color picker, light/dark/AMOLED-black modes, density options, and a couple of icon themes — is a strong retention and identity feature, and a clean Pro upsell, while staying fully additive to the existing theme code. Personalization converts a tool into 'my tool': users who customize an app invest in it and churn less, and on Android, theming is an expected mark of a polished, premium-feeling utility.

Theming is a recognized differentiator and monetization surface in this exact category. Solid Explorer is well known for its customizable themes and accent colors; many file managers gate premium themes behind their paid tier. Material You dynamic color is also a quality signal modern Android users specifically look for. Offering AMOLED-true-black appeals directly to the power-user and privacy-conscious personas (battery, OLED, minimalism), and a small set of curated icon/density themes gives the Pro tier (initiative #1) a tangible cosmetic upsell that costs nothing at the filesystem layer.

Technical approach: extend ui/theme/Theme.kt and Color.kt to accept a runtime theme configuration (accent seed color, dark-mode mode including AMOLED black, density) sourced from a Flow. Persist choices in data/preferences/SettingsDataStore.kt and expose them through feature/settings/SettingsScreen.kt + SettingsViewModel.kt with a live-preview accent picker and Material You toggle. Apply the configuration at the top of the Compose tree in MainActivity.kt so the whole app recomposes against the chosen theme. Premium themes/icon packs are simply gated by Feature (entitlement, initiative #1); the base theme stays free.

POINT 3 — IMPLEMENTATION PROMPT

```
Extend ui/theme/Theme.kt:
  Accept runtime ThemeConfig (accentSeed, darkMode incl. AMOLED, density)
  Material You dynamic color support
Extend ui/theme/Color.kt: accent-seeded palettes + true-black surfaces

Persist in data/preferences/SettingsDataStore.kt (exposed as Flow)

feature/settings/SettingsScreen.kt + SettingsViewModel.kt:
  Accent-color picker (live preview), light/dark/AMOLED, density,
  icon-theme selector, Material You toggle

MainActivity.kt: collect ThemeConfig, apply at top of Compose tree
Premium icon/density themes gated by Feature (entitlement #1)
Additive: base theme stays free; no regression
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Retention / identity	Customized app = lower churn
Pro cosmetic upsell	Premium themes/icon packs
Persona fit	AMOLED black for power/privacy users
Reuse	Extends existing JupiterTheme

ESTIMATED VALUATION IMPACT

+€70k-€150k

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design personalization & theming on top of the existing JupiterTheme. Cover: (1) retention/identity + Pro upsell rationale citing Solid Explorer themes and Material You expectations, (2) extend ui/theme/Theme.kt + Color.kt for runtime ThemeConfig (accent seed, dark/AMOLED, density, Material You), persist via SettingsDataStore, SettingsScreen/ViewModel live-preview picker, apply in MainActivity, premium themes gated by Feature, (3) additive non-regression scope, (4) retention/upsell valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

BANK 10 Share & Interop Hub (Receive "Save to Jupiter", Quick-Share, SAF DocumentsProvider)

POINT 2 — DESCRIPTION & MARKET EVIDENCE

Jupiter is currently a destination the user opens; it should also be the hub other apps route through. Registering as an ACTION_SEND / SEND_MULTIPLE receive target ('Save to Jupiter'), adding a Storage Access Framework DocumentsProvider so any app's file picker can browse Jupiter's storage and vault, and adding quick-share/send-to-device, makes Jupiter appear inside the share sheet and system file picker of every other app. That drives virality (each share is a discovery surface) and stickiness (other apps now depend on Jupiter as a provider), and it is additive — implemented through the manifest plus new components, with the existing data/file layer doing the actual work.

Deep OS interop is exactly how the leaders embed themselves. Files by Google and Samsung My Files register broadly as share targets and document providers; a SAF DocumentsProvider in particular lets Jupiter surface inside Gmail attachments, document editors and any app that opens the system picker — turning Jupiter into infrastructure rather than a standalone app. For a privacy-first manager, being the receive target for 'Save to Jupiter' (including straight into the encrypted vault) is also a differentiated, trust-led entry point that no ad-supported competitor can frame as cleanly.

Technical approach: declare ACTION_SEND and ACTION_SEND_MULTIPLE intent-filters and a in AndroidManifest.xml. Add a feature/share package with a ReceiveShareActivity / ReceiveShareScreen (Compose) that lets the user pick a destination — including the vault — and writes via the existing data/file pipeline. Add a data/saf package implementing JupiterDocumentsProvider (extending DocumentsProvider) over the same file layer so other apps can browse and open Jupiter content through the system picker. Quick-share / send-to-device builds on the existing NanoHTTPD Wi-Fi server and transfer stack for nearby device handoff.

POINT 3 — IMPLEMENTATION PROMPT

```
AndroidManifest.xml:
<intent-filter> ACTION_SEND + ACTION_SEND_MULTIPLE (Save to Jupiter)
<provider> for the SAF DocumentsProvider

Create feature/share/ReceiveShareActivity.kt + ReceiveShareScreen.kt
  Destination picker incl. encrypted vault
  Write incoming items via existing data/file pipeline

Create data/saf/JupiterDocumentsProvider.kt (extends DocumentsProvider)
  Expose Jupiter storage to other apps' system file pickers
  Back onto the same data/file layer

Quick-share / send-to-device:
  Reuse NanoHTTPD Wi-Fi server + data/transfer stack
  Additive via manifest + new components; data/file reused
```

POINT 4 — VALUE INCREASE ESTIMATION

Metric	Impact
Virality	Appears in every app's share sheet
Interop stickiness	SAF provider in system file pickers
Trust entry point	'Save to Jupiter' direct to vault

Reuse	data/file + NanoHTTPD Wi-Fi server + transfer
ESTIMATED VALUATION IMPACT	
+€100k-€200k	

POINT 5 — REGENERATION PROMPT

Analyze Jupiter and design a Share & Interop Hub. Cover: (1) virality/stickiness rationale citing Files by Google/Samsung My Files share-target and document-provider embedding, (2) AndroidManifest ACTION_SEND/SEND_MULTIPLE intent-filters + <provider>, feature/share ReceiveShareActivity/Screen writing via data/file incl. vault, data/saf JupiterDocumentsProvider extending DocumentsProvider over data/file, quick-share reusing NanoHTTPD Wi-Fi server + transfer, (3) additive manifest + new components scope, (4) virality/stickiness valuation impact, (5) regeneration meta-prompt. Point 1-5 structure.

COMPLETE VALUE MATRIX — All Growth Initiatives

Ran k	Initiative	Estimated Impact
1	Jupiter Pro — Monetization & Entitlement Engine	+€350k-€700k
2	Real Cloud Storage — Google Drive / Dropbox / OneDrive (OAuth + REST)	+€220k-€420k
3	Localization & Global Reach (12+ languages)	+€120k-€260k
4	AI Pro Suite (Claude) — Summarize, Smart-Rename, Auto-Organize, Semantic Search	+€160k-€320k
5	Home-screen Widgets, App Shortcuts & Quick-Settings Tile	+€90k-€180k
6	Privacy-first Opt-in Product Analytics & Crash Reporting	+€80k-€170k
7	Activation & Retention Loop (Onboarding Funnel, What's-New, Rating, Re-engagement)	+€90k-€190k
8	High-performance Search Index (Room FTS) + Saved Searches + Content Search	+€110k-€220k
9	Personalization & Theming (Material You Accent, Icon/Density Themes, Custom Home)	+€70k-€150k
10	Share & Interop Hub (Receive "Save to Jupiter", Quick-Share, SAF DocumentsProvider)	+€100k-€200k
TOTAL COMBINED POTENTIAL		+€1.39M-€2.81M

Baseline Valuation: €140k-€320k

→ Target Valuation: €1.4M-€3.2M (8-10x)